Chapter 10: Objects

Suppose we are writing a small gradebook. For each student, we need to keep a name and an exam score together. Ada scored 87. One way to store her entry is a list:

```
student = ["Ada", 87];
print(student[0]);
print(student[1]);
```

A **list** stores values in order. Brackets select a value by its position, counting from zero, so these statements print `Ada`, then `87`. The program works, but a reader has to remember that position zero means the name and position one means the exam score. Neither meaning is visible in `student[1]`.

An **object** lets us name those parts of the entry:

```
student = {"name": "Ada", "score": 87};
print(student.name);
print(student.score);
```

The output is still `Ada`, then `87`. Now `student.score` says which fact we are reading. Each named part is a **field**: a string **key**, such as `"score"`, associated with a value, such as `87`. Access by key makes this an associative collection. (1)

Positional record	Named record
`["Ada", 87]`	`{"name": "Ada", "score": 87}`
`student[0]`	`student.name`
`student[1]`	`student.score`

In this chapter, an object is a **mutable record** with string keys. Mutable means its fields can change while it remains the same object. A record groups related facts about one thing. There is no class declaration, and these objects have no methods or inheritance. Lists still fit ordered sequences, such as a student's exam scores; objects give names to the different facts about that student.

Constructing an Object

An object literal contains comma-separated key-value pairs between braces. Empty braces construct an empty object. Each evaluation constructs a new value.

```
empty = {};
field = "score";
student = {"name": "Ada", field: 87, "passed": true};
```

A key can be any expression, but its evaluated value must be a string. Values can be any Vertex values, including lists and other objects. No trailing comma is allowed, matching the list literal grammar.

Expression	Result
`{"na" + "me": "Ada"}`	One field named `name`
`{1: "Ada"}`	Key-type error
`{"score": 87, "score": 90}`	Duplicate-key error
`{"score": 87,}`	Syntax error

Duplicate checking uses the evaluated key. Two different expressions that both produce `"score"` still collide. Construction reports the error rather than silently discarding one value.

Reading and Writing Fields

Brackets evaluate a key expression. Dot notation spells a literal key using an identifier.

```
student = {"name": "Ada", "score": 87};
field = "score";
print(student["score"]);
print(student.score);
print(student[field]);
```

All three reads print `87`. `student.field` would instead ask for the key `"field"`; it would not evaluate the variable `field`. Keys containing spaces or punctuation need brackets: `student["final score"]`.

```
student.score = student.score + 3;
student.passed = true;
print(student);
```

The result is `{"name": "Ada", "score": 90, "passed": true}`. Updating a field changes the existing object. Writing a new key adds a field. Reading a missing key raises a missing-key error; it does not create a field or return a default. List assignment still requires an existing position. Object fields can grow because a record may acquire another named fact.

Nested Data and Sharing

A student may have several exam scores. Store those scores as a list inside the student object, then put the student objects into a class list:

```
students = [
    {"name": "Ada", "scores": [87, 93]},
    {"name": "Lee", "scores": [82, 90]}
];
students[0].scores[1] = 95;
print(students[0].scores);
```

This prints `[87, 95]`. Each suffix selects a value from the preceding one. There is no special operation for a list of records.

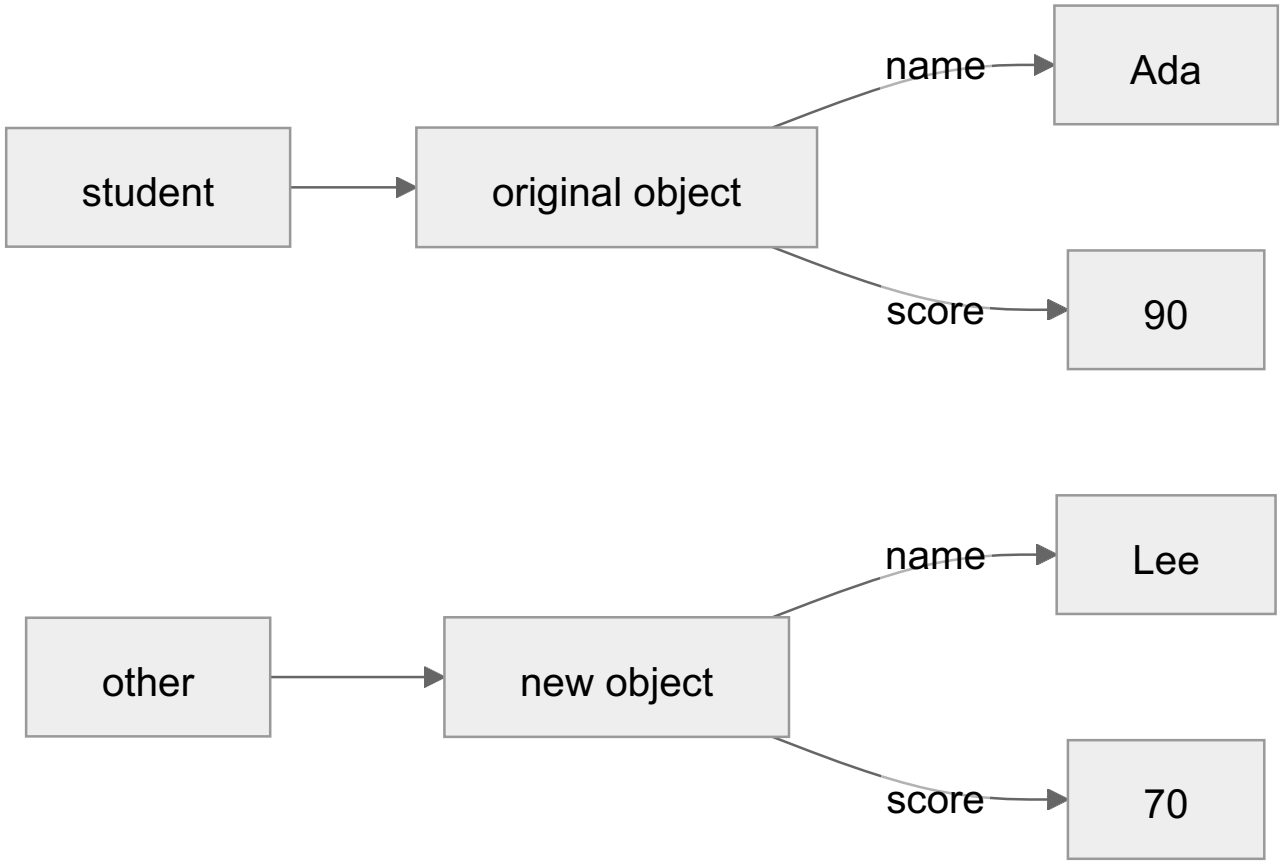
Objects follow the sharing rule from Lists and Indexing: assigning another name to a record shares that record. (2) Updating a field changes the record; rebinding a name leaves the record alone. The new part is selecting a field by key instead of a list position:

```
student = {"name": "Ada", "score": 87};
other = student;
other.score = 90;
print(student.score);
other = {"name": "Lee", "score": 70};
print(student.score);
```

Both prints produce `90`, but the shared score did not start there. It starts at `87`; `other.score = 90` updates the one record reached through both names. The last assignment rebinds `other` to a new record whose score is `70`. It does not change the original record again.

After this statement	`student.score`	`other.score`
`other = student`	87	87
`other.score = 90`	90	90
`other = {"name": "Lee", "score": 70}`	90	70

The diagram shows the final state, after `other` has been rebound. Variable names use unlabeled arrows to reach their objects. Object field names label outgoing arrows, which lead to value boxes. The field names stay on the arrows throughout these object diagrams.



Nested values follow the same rule. Putting an existing list into two fields does not create two lists. Mutation through either field reaches the shared list.

Equality and Display

Objects are equal when they have the same keys and recursively equal values. Insertion order does not affect equality.

```
a = {"name": "Ada", "score": 87};
b = {"score": 87, "name": "Ada"};
print(a == b);
print(a);
print(b);
```

This prints:

```
true
{"name": "Ada", "score": 87}
{"score": 87, "name": "Ada"}
```

Printing preserves insertion order. Updating an existing field leaves its position alone; adding a field puts it at the end. `string(a)` uses the same object representation as `print(a)`, including quoted keys and string values.

Equality keeps Vertex's scalar distinctions at every depth. For example, `{"ok": true} == {"ok": 1}` is `false`. An object and a list are different kinds of value even when both are empty.

Deep Copying and Cycles

To change nested data independently of the original, use `copy` to allocate new mutable containers.

How much of the reachable data should be new? An object can contain a list, which can contain another object. Copying just the first container does not make everything inside it independent.

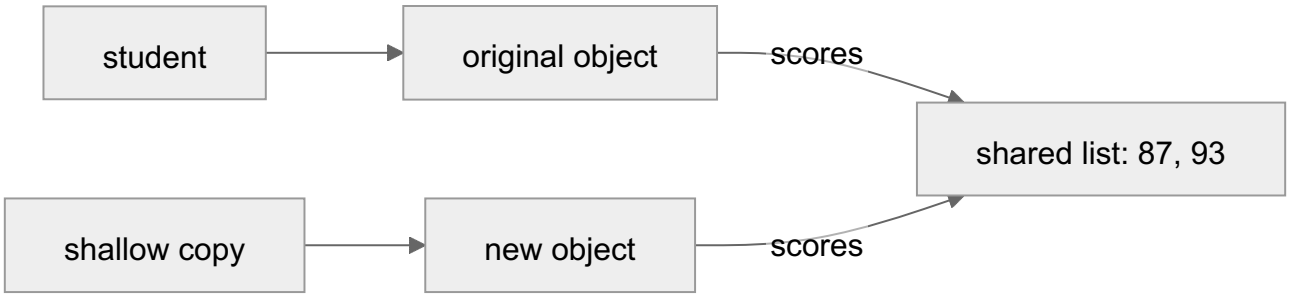
Shallow and Deep Copies

Start with a student record containing a scores list:

```
student = {"name": "Ada", "scores": [87, 93]};
```

The record and the list are two separate mutable values. The record's `scores` field refers to the list; it does not contain a private duplicate of that list. In the diagrams below, each box labeled object or list is a separate container. An unlabeled arrow follows a variable name; a labeled arrow follows an object field. Keys label arrows, while boxes show containers or values. The copy diagrams show only the `scores` field of the student record, so the nested sharing is easier to follow.

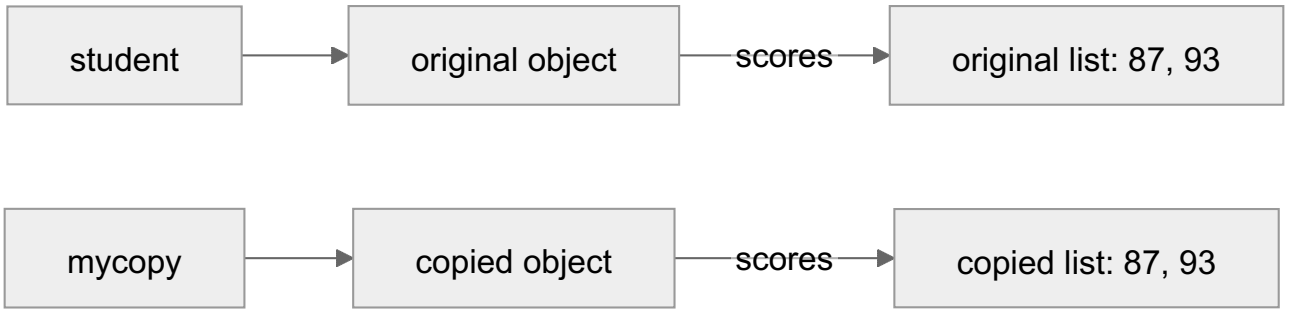
A **shallow copy** makes a new outer container and keeps references to the original nested values. (3) This is what a shallow copy of the student record would look like:



There are two objects, but only one scores list. Changing the new object's `name` field would leave the original object's name alone. Changing an element through its `scores` field would change the list both objects see. A new outer object is not enough to isolate a nested mutation.

This diagram describes a copying strategy, not a second Vertex object-copy operation. Vertex's `slice` and `tail` make shallow copies of sequences; `copy` makes a **deep copy**, recursively copying nested mutable values. (3)

```
mycopy = copy(student);
```



Now there are two objects and two lists. The copied object's `scores` field leads to the copied list. No arrow from the copied data leads back to an original mutable container.

```
mycopy.scores[1] = 100;
print(student.scores);
print(mycopy.scores);
```

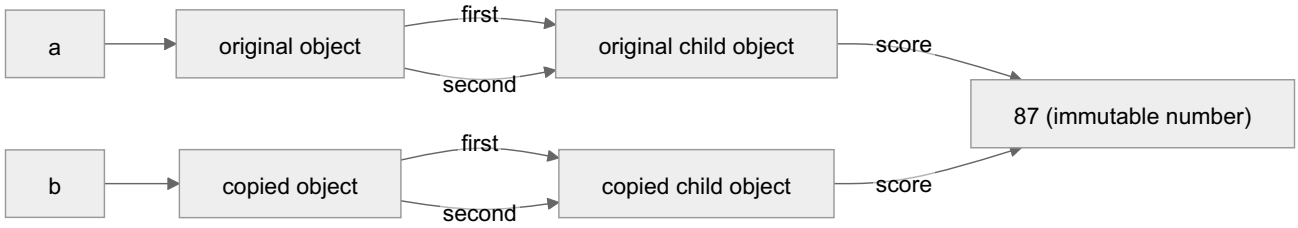
This prints `[87, 93]`, then `[87, 100]`. The mutation reaches only the copied list. Unlike a shallow copy, this copy separates the nested mutable containers too. Immutable strings, numbers, and booleans can remain shared safely; there is nothing inside those values that the copy could change.

Keeping Shared Children Shared

Deep copying does not mean making a fresh container for every arrow. Suppose two fields refer to one child object:

```
child = {"score": 87};
a = {"first": child, "second": child};
b = copy(a);
```

The original has one child, reached through either field. The copy also has one child, reached through either copied field. It is a new child, separate from the original:



Both `score` arrows lead to the immutable number `87`. The copied child object is new, but the number does not need duplicating. Changing a score field later changes that field's reference; it does not change the number `87`.

```
b.first.score = 90;
print(b.second.score);
print(a.first.score);
```

This prints `90`, then `87`. The copy preserves the relationship between `first` and `second`: they still reach the same child. The original and the copy no longer share mutable containers. Making two unrelated copied children would change the meaning of the structure.

Copying a Self-Reference

A cycle is another relationship to preserve. Here the `self` field refers to the very object that owns the field:

```
a = {};
a.self = a;
b = copy(a);
```

There is only one original object. Following `a.self` returns to that object; following `a.self.self` does the same thing again. Copying it must produce one new object whose `self` field returns to the new object:





The two loops are separate. `b.self` does not point to `a`, and copying does not create an endless chain of new objects.

```
print(a);
print(a == b);
b.score = 90;
print(b.self.score);
print(a);
```

This prints:

```
{"self": {...}}
true
90
{"self": {...}}
```

The first print uses `{...}` to mark a return to an object already being printed. The equality test is `true` because, at that point, both objects have only a `self` field leading back to themselves. Equality compares their contents, not whether the two names designate the same object.

Then `b.score = 90` adds a field to the copied object. Reading `b.self.score` first follows the loop back to that same copied object and then reads its new score. The last print confirms that `a` still has only its self-reference. After adding the field to `b`, the two objects are no longer equal.

Remembering What Has Been Copied

A **memo table** is just a dictionary that records, for something already handled, what the result was – so that handling the same thing again means looking up that result instead of redoing the work. `vertex_copy` needs one because a plain recursive copy, with nothing to remember what it has already done, gets two different cases wrong.

The first is the shared-child example above: without a memo table, each of `a`'s two fields would trigger its own fresh copy of `child`, and the copy would end up with two unrelated child objects where the original had one. The second is self-reference: copying `a.self` means copying `a`, which means copying `a.self` again, forever, with no base case to stop at.

A memo table fixes both at once. `vertex_copy` maps each original container's identity to its copy. The mapping lives in this one table, not on the container itself: `child` does not hold a reference to its own copy, and has no idea the table exists. `vertex_copy` computes `id(child)` and looks that up in `memo` from the outside; the pointer to the copy lives in the table, not in `child`. Before copying any of a container's child values, it first allocates the new, still-empty container and records that mapping. For the shared-child example, the first encounter with `child` creates a copied child and records it; the second encounter finds that same copy already in the table and reuses it instead of making a second one. For the self-reference, the original object is already in the table by the time its `self` field is reached, because its copy was recorded before any field was copied. The lookup returns that new object, still being filled in, which closes the copied loop instead of recursing again. There is one table per `copy()` call, not one per container: it is created once, at the start of that call, and covers every container reached while copying that one value, including paths that cross between lists and objects.

Comparing and printing collections need their own bookkeeping. `vertex_equal` tracks pairs of identities currently being compared. `vertex_repr` tracks ancestors on the current display path and writes `{...}` for a repeated object or `[...]` for a repeated list. These helpers have different jobs: comparison checks corresponding contents, display makes cycles readable, and copying reconstructs the original relationships using new mutable containers.

Types and Supported Operations

Operation	Object behavior
`type({})`	Returns `"object"`
`string(obj)` and `print(obj)`	Display fields in insertion order
`copy(obj)`	Deep copy, preserving internal sharing and cycles
`obj == other`, `obj != other`	Structural equality and inequality
`number(obj)`, `boolean(obj)`	Type error
`length`, `head`, `tail`, `slice`	Objects are not accepted

Arithmetic and ordering comparisons do not accept objects. A Python dictionary is the storage mechanism, not permission to expose every Python dictionary operation as a Vertex feature.

Parsing the New Syntax

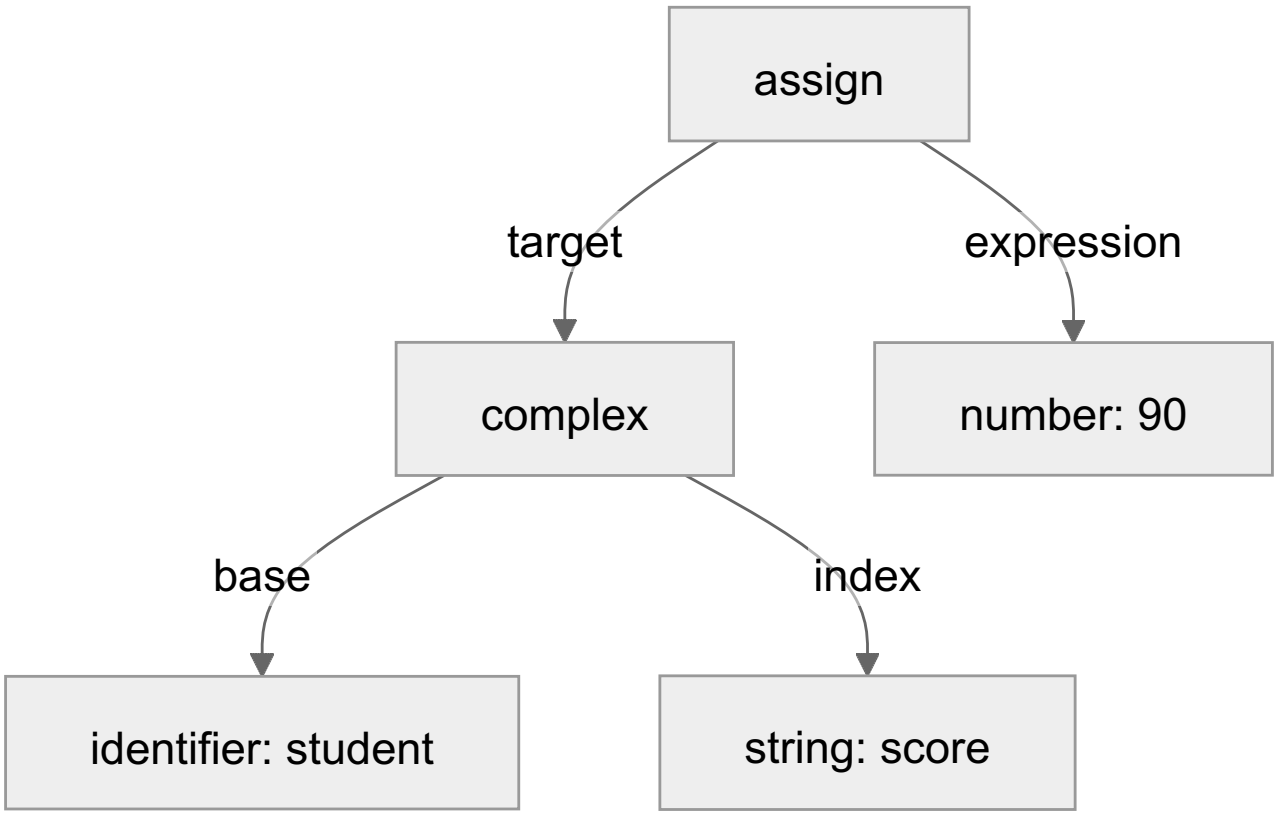
The tokenizer adds `:`. The parser recognizes an object literal where it expects an expression. Braces used for statement blocks are still handled by the block parser. After `if (condition)`, a block is required; after an assignment's equals sign, braces can introduce an object literal.

The grammar in Extended Backus-Naur Form (EBNF) is:

```
object_expression ::= "{" [ field { "," field } ] "}"
field             ::= expression ":" expression
index_suffix      ::= "[" expression "]" | "." identifier
```

`parse_object_expression` stores fields as an ordered list of key-expression and value-expression pairs. It cannot use a dictionary of evaluated fields: parsing has not evaluated keys, and duplicate keys must remain visible until the evaluator checks them.

`parse_complex_suffixes` converts `student.score` into the same abstract syntax tree (AST) as `student["score"]`. Both use the existing `complex` node. The assignment `student.score = 90` has this tree. Unlike the runtime object diagrams, these boxes represent syntax nodes. The `string: score` box holds a literal expression; it is not an object with a field inside it:



Parsing validates the punctuation and the identifier after a dot. Evaluation checks the selected value's type and the computed key's type.

Evaluation Order and Status

The evaluator returns a pair, `(value, status)`. A status of `None` means evaluation completed normally. `exit(7)` instead returns `(7, "exit")`; an enclosing operation must pass that result outward rather than keep evaluating. The same rule applies to list elements, object fields, and assignment values.

Object construction proceeds field by field, from left to right:

1. Evaluate the key and check its returned status.
2. Require a string key and reject a duplicate.
3. Evaluate the value and check its returned status.
4. Store the field in the new object.

A duplicate or invalid key is rejected before its value expression runs. For either a read or an indexed assignment, the base is evaluated before the key. Assignment validates the destination before evaluating the right-hand value. This keeps a bad key from triggering unrelated right-hand side effects.

`exit` cannot actually sit inside `student.score = exit(7)` or `{"score": exit(7), "name": input()}`: it cannot nest inside any expression, only stand alone as a statement. Built directly as AST nodes rather than parsed from source, the same claims hold: assigning `exit(7)` into `student.score` exits with status `7` and leaves the score unchanged. Likewise, a field whose value is `exit(7)`, followed by a field whose value is `input()`, exits before the input call ever runs and never assigns the object at all. Earlier side effects are not rolled back. Status propagation stops later work; it is not a transaction mechanism.

Objects and Environments

Objects and environments both use plain Python dictionaries in this interpreter. Their lookup rules stay separate.

Object field read	Identifier lookup
Starts from an evaluated data value	Starts from the current environment
Requires a string key	Uses an identifier name
Reads the selected dictionary directly	May follow enclosing-environment links
Missing key raises an error	Missing name may be sought outward

An object's fields never pass through `global_environment` or the identifier lookup walk. A field named `"$PARENT"` is ordinary data:

```
grandparent = {"name": "Mae", "birthplace": "Kent"};
parent = {"name": "Lee", "$PARENT": grandparent};
print(parent["$PARENT"].birthplace);
```

This prints `Kent` because the program explicitly follows the stored field. Reading `parent.birthplace` instead raises a missing-key error. It does not search `grandparent`. The environment's own `PARENT_KEY` remains the plain string `"$PARENT"`; its safety comes from keeping these lookup paths separate. String-keyed dictionaries also keep the reference implementation straightforward to reproduce in another host language.

Verification

`code/objects.v` is a runnable example of construction, field mutation, nesting, copying, and cycles. Run it with `code/vertex code/objects.v` from this chapter's directory. `bash code/run-tests.sh` checks the interpreter and both the inherited behavior and the new object rules. The object tests cover errors as well as successful programs, including evaluation order and interrupted mutations.

Considerations

1. What mistake could a missing-field default hide? Where would an explicit default operation be useful without changing ordinary field reads?
2. Why does updating a field preserve its display position even though equality ignores field order?
3. When would preserving shared children inside a deep copy matter? What would change if the copy duplicated every occurrence independently?
4. How could a different host representation keep the same object semantics? Which rules must remain independent of the host's dictionary operations?

Practice

1. Starting with `student = {"score": 87};`, add a `passed` field whose value is whether the score is at least `60`.
2. Predict the final values of `a.score` and `b.score`:

   ```
   a = {"score": 87};
   b = a;
   b.score = 90;
   b = {"score": 70};
   ```

3. Write a program that copies `{"scores": [80, 90]}`, changes the copy's first score to `100`, and prints both score lists.
4. Explain why `{"a": 1, "a" + "": input()}` reports an error without asking for input.

5. Create an object that refers to itself through a `self` field. Copy it and explain which object the copied `self` field refers to.

Answers to the Practice Questions

1. `student.passed = student.score >= 60;` adds the field with value `true`.
2. `a.score` is `90`; `b.score` is `70`. Mutation changed the shared object. Rebinding `b` then gave it a different object.
3. One solution is:

   ```
   original = {"scores": [80, 90]};
   mycopy = copy(original);
   mycopy.scores[0] = 100;
   print(original.scores);
   print(mycopy.scores);
   ```

   This prints `[80, 90]`, then `[100, 90]`.
4. Both key expressions produce `"a"`. Duplicate checking happens before the second value expression is evaluated, so `input()` never runs.
5. `a = {}; a.self = a; b = copy(a);` makes `b.self` refer to `b`. The memo table records the new object before recursively copying the original's fields.

References

1. Wikipedia contributors. "Associative array." https://en.wikipedia.org/wiki/Associative_array
2. Wikipedia contributors. "Aliasing (computing)." https://en.wikipedia.org/wiki/Aliasing_(computing)
3. Python Software Foundation. "copy - Shallow and deep copy operations." Python documentation. https://docs.python.org/3/library/copy.html